```
require "minitest/autorun"

class TestWordsFromString < Minitest::Test
  def test_empty_string
    assert_equal([], words_from_string(""))
    assert_equal([], words_from_string("     "))
  end

  def test_single_word
    assert_equal(["cat"], words_from_string("cat"))
    assert_equal(["cat"], words_from_string("  cat   "))
  end

  def test_many_words
    assert_equal(
      ["the", "cat", "sat", "on", "the", "mat"],
      words_from_string("the cat sat on the mat")
    )
  end

  def test_ignores_punctuation
    assert_equal(
      ["the", "cat's", "mat"],
      words_from_string("<the!> cat's, -mat-")
    )
  end
end
```

produces:

```
Run options: --seed 11339
# Running:

....
Finished in 0.000367s, 10899.1825 runs/s, 16348.7738 assertions/s.
4 runs, 6 assertions, 0 failures, 0 errors, 0 skips
```

The test starts by requiring the source file containing our words_from_string method, along with the unit test framework itself. It then defines a test class. Within that class, any methods whose names start with test are automatically run by the testing framework. The results show that four test methods ran, successfully executing six assertions.

We can also test that our count of word frequency works:

tut_containers/word_freq/test_count_frequency.rb

```
require_relative "count_frequency"
require "minitest/autorun"

class TestCountFrequency < Minitest::Test
  def test_empty_list
    assert_equal({}, count_frequency([]))
  end

  def test_single_word
    assert_equal({"cat" => 1}, count_frequency(["cat"]))
  end

  def test_two_different_words
    assert_equal({"cat" => 1, "sat" => 1}, count_frequency(["cat", "sat"]))
  end
```